GridIQ

Senior Capstone Proposal

Stan Pierre

Computer Science Department
Endicott College

March 13, 2026

Contents

Abstract

This project proposes the development of GridIQ, an intelligent football analytics tool designed for high school and amateur coaches. The system will allow users to visualize and store football play data, input formation and result information, and use artificial intelligence to identify player and team tendencies from video footage. The goal of this project is to create a functional prototype that combines data visualization, computer vision, and machine learning to enhance play analysis and scouting efficiency.

1. Introduction and Background

Football is a game of patterns. “Game tendencies” refer to the predictable habits a team shows in specific situations. For example, a team might run the ball 90% of the time when it is “2nd down and 1 yard to go.” Coaches use these tendencies to predict opponent strategies during game preparation and to anticipate play calls during the game. Currently, identifying these patterns requires hours of manual film study and data entry. Coaches usually track this information by hand during games or review it manually afterward. This process is slow, inconsistent, and difficult to share across a staff.

Larger programs use software such as Hudl [1] or Catapult [2] to automate parts of this workflow. Hudl provides play tagging and breakdowns but requires a paid subscription that can reach several hundred dollars per team each season. Catapult uses wearable devices and GPS tracking, which raises the cost into the thousands and depends on hardware high school teams typically do not own. Professional tools such as NFL Next Gen Stats [3] use RFID chips and advanced tracking systems that are completely out of reach for non professional programs.High school and smaller collegiate teams need something more practical.A tool that works in a normal web browser, does not require special equipment, and supports simple workflows coaches already use. GridIQ is designed to meet that need. It lets users enter plays in structured form, view formation diagrams generated from those entries, and review common tendencies such as run/pass ratios and average gains by formation. Coaches can also upload game film and link plays to specific moments in a video to make post game review faster. The system processes all analytics offline and after games rather than in real time. This keeps the design simple and matches how high school teams operate. They review film throughout the week and use that information to plan the next game. GridIQ focuses on delivering clear visual summaries without the cost or complexity of professional platforms.

2. Requirements

This section describes what the completed version of GridIQ should look like. The system is a web application built with a React frontend, a Flask backend, a PostgreSQL database. It does not require an internet connection for most features but includes limited online features for basic authorization.

The primary requirement for GridIQ is to provide a robust, offline capable interface for data collection and analysis. The system should allow users to manually input play data through a secure, locally hosted web interface. The application should run within Docker containers on the user’s machine, ensuring that all database interactions and business logic function without an active internet connection. This offline capability is critical for coaches working in press boxes or buses where Wi-Fi is unreliable. Additionally, the system should implement a basic authentication mechanism to secure team data, ensuring that unauthorized users cannot access or change sensitive playbook information stored on the device.

Once data is captured, the system should provide the analytical feedback through a dynamic dashboard. The application should query the local PostgreSQL database to generate tendency reports, such as visualizing the ratio of run plays versus pass plays in specific down-and-distance situations. These visualizations should be interactive, they should allow the user to filter results by specific formations or field zones. To further aid in game planning, this system should include a “Formation Diagram” tool that shows a visual representation of the offensive alignment based on the input data, reducing the need for coaches to draw plays by hand.

Besides the basic data entry, the system should include artificial intelligence to automate the scouting process. The application should utilize computer vision (specifically the YOLOv8 [4] model) to detect player positions from uploaded game footage. Once players are detected, the system should attempt to classify the offensive formation automatically, automating the data entry part. As a future enhancement, the system should also offer a "Predictive Mode" that analyzes historical opponent data to offer the most likely upcoming play, as well as an export feature to generate printable PDF scouting reports for players to study offline or by hand.

Requirement Name	**Complexity**	**Priority**	**Notes**
Play Input System	Low	1	Must have
Play Visualization	Medium	1	Must have
Usability and Accessibility	Low	1	Must have
Local Operation	Low	1	Must have
Performance Efficiency	Medium	2	Must have
Data Security and Privacy	High	2	Must have
Tendency Analysis Dashboard	Medium	2	Must have
Video Upload and Linking	Medium	2	Want to have
Data Export	Low	3	Want to have
User Authentication and Data Management	Medium	3	Want to have
Scalability and Extensibility	High	3	Want to have

Table 1: Requirements summary with complexity, priority, and classification.

As shown in Table 1, the highest priority requirements are shown as must have and form the core of the whole project. Want to have requirements extend the system with video functionality, advanced security, and export features, and will be completed if time permits during the capstone phase.

3. Architecture

Coaches access the system through a web browser, but every part of the application executes locally. This approach removes the need for internet connectivity and matches the workflows of high school teams that often analyze film in locker rooms, classrooms, or fields without reliable Wi-Fi. The architecture includes three main layers: a React [5] frontend, a Flask [6] backend, and a PostgreSQL [7] database. These components communicate through local REST API calls and share data through Docker [8] volumes. The system also stores uploaded video files on the local machine, ensuring that film review and tagging work without network access.

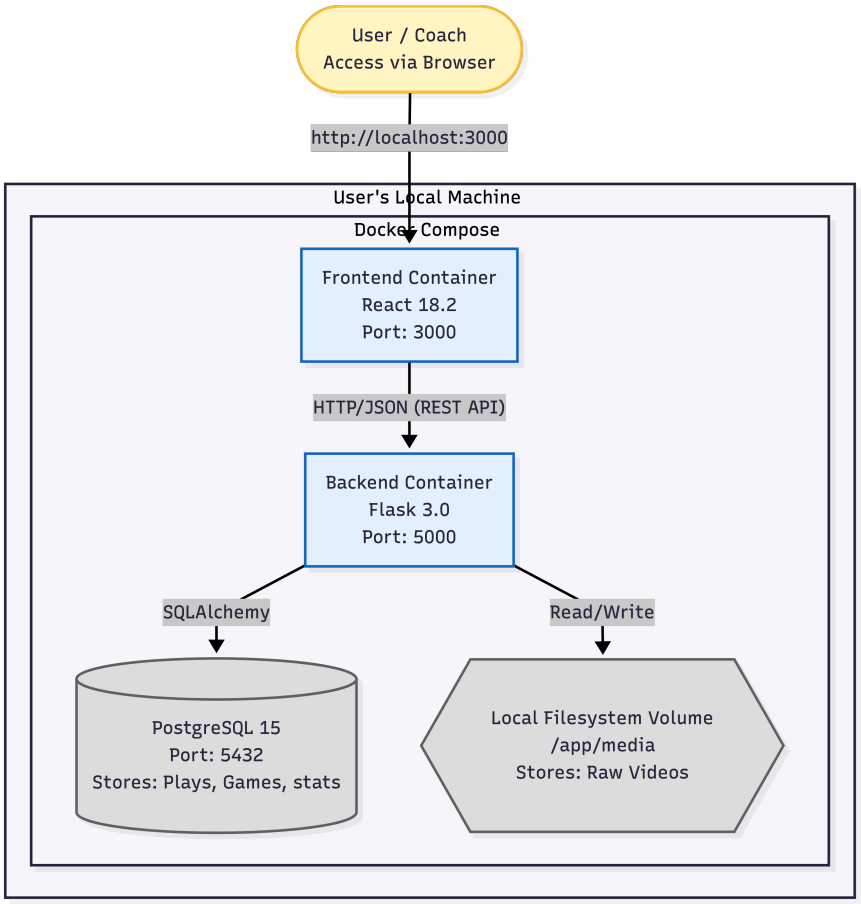

Figure 1: System architecture showing component interactions and data flow.

As seen in the Figure 1 the presentation Layer (**Frontend**) is built using React [5] and TypeScript [9]. This layer handles all user interactions, such as data entry and chart rendering. It runs in its own container and communicates with the backend solely through HTTP requests to local host. The Application Layer (**Backend**) serves as where the logic is, built with Python Flask. This container exposes a API that approves of incoming data, handles business logic, and executes the statistical algorithms required for the tendency analysis. Python was selected for the available data science libraries, which will be needed for the future integration of the YOLOv8 computer vision models. The Data Layer (**Database**) utilizes PostgreSQL [7]. While a simple file-based storage solution (like JSON or SQLite) is often standard for offline apps. A relational database was used to handle the complex analytical queries required by the application. Football analytics often require tough findings like finding "pass plays on 3rd down between the 20 and 40-yard lines" which PostgreSQL handles with significantly higher performance and data integrity than flat files. The data flow within this architecture follows a request and response cycle. When a user inputs a play, the React frontend packages the data into JSON and sends a `POST` request to the Flask API. The Flask application authenticates the request, validates the data schema using a model, and uses the SQLAlchemy [10] to keep the record to the PostgreSQL container. For data visualization, the process is reversed. The frontend requests the statistics, which the backend calculates via SQL queries before returning the results to show. This separation ensuring that heavy data processing never blocks the user interface.

The most significant architectural decision was the choice to containerize the application rather than build a native executable. The tradeoff is a slightly more complicated installation process, as the user must have Docker installed. But, this is outweighed by the benefit of consistency, the PostgreSQL database and Python dependencies run exactly the same on any machine, regardless of the host operating system. Additionally, this architecture takes away the frontend from the backend. If we decide to move to a cloud hosted model in the future, the architecture requires minimal changes. Simply move the containers from the local machine to a cloud server, whereas a native desktop app would require a complete rewrite.

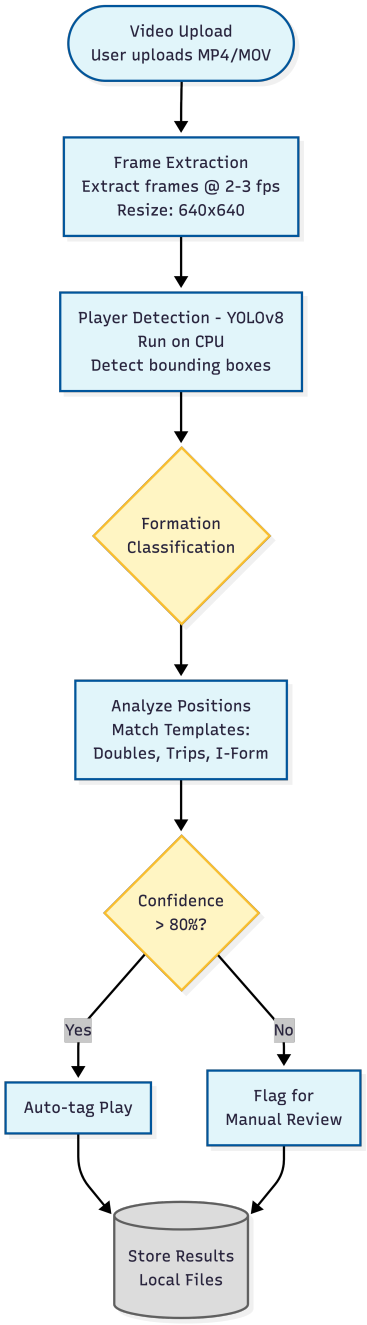

Figure 2: System architecture showing video component and video flow.

As seen the Figure 2, when a user uploads a video file, the system immediately begins processing in the background. The Video Processor component first extracts frames from the video at a slow rate, 2-3 frames per second. Each frame is preprocessed by resizing to 640x640 pixels (the input size of YOLOv8). The player detection stage uses YOLOv8 to identify player bounding boxes in each frame. The model runs on CPU by default, making it accessible to coaches using standard laptops without dedicated graphics hardware. A typical 10-minute game film segment processes in approximately 20-30 minutes on a modern laptop, which coaches can run overnight or during the school day. After detecting players, the formation classifier tries to identify the formation based on player positions. This counts players in different zones of the field and compares the arrangement to known formations as “Doubles Right” or “Trips Left”. If the classifier achieves high confidence (above 80%), it automatically tags the play with the detected formation. If confidence is low, the play is does not get tagged. Then it is looked at for coaches to review. They then get stored to the backend.

4. Interfaces

GridIQ runs as a web application in the browser but executes entirely on the local machine. The interface uses a tab-based navigation system that gives coaches quick access to the four main views: Play Input, Dashboard, Opponent Scouting, and Games. This keeps critical functions like entering new plays or checking tendencies just one click away.

4.1. User Interface Screens

The Play Input screen is the main data entry point. It uses a form with drop down menus for fields like formation, down, and play type to prevent typos and keep data consistent. There is also an “Advanced Details” toggle that expands to show optional fields like defensive front, coverage, and blitz without cluttering the basic form. Below the form, a table shows all previously entered plays with the most recent at the top. Figure 3 shows how this screen looks like.

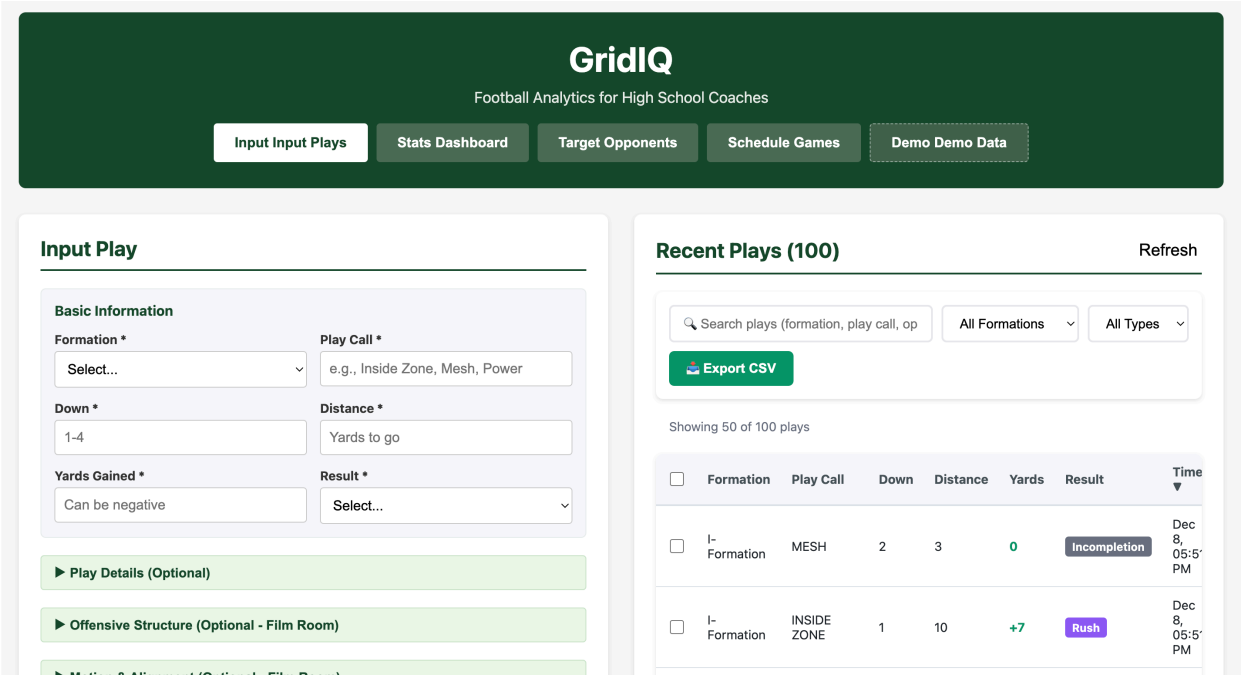

Figure 3: Play input form showing basic fields and expandable advanced options with play history table below

The Dashboard serves as the analytics hub. At the top, the Tendency Analyzer scans all plays and displays warning cards when it finds predictable patterns. For example, if you run the ball 85% of the time on first down, it shows a red warning card. Below that, the Play Predictor lets coaches select a situation (formation, down, distance) and get an AI prediction of whether the play will be run or pass, with a confidence score shown as a colored circle. The Figure 4 shows stat cards for total plays, yards, and touchdowns, followed by bar charts for plays by down and formation performance tables.

Figure 4: Analytics dashboard showing tendency warnings, play predictions, and statistical breakdowns

The heat map visualization appears on the dashboard as well. It shows a football field divided into three zones (left, middle, right) with each zone color-coded based on success. Green means high success in that direction, yellow is average, and red indicates poor performance. Users can filter by run or pass plays and toggle between showing average yards or success rate percentage. This is shown below in Figure 5.

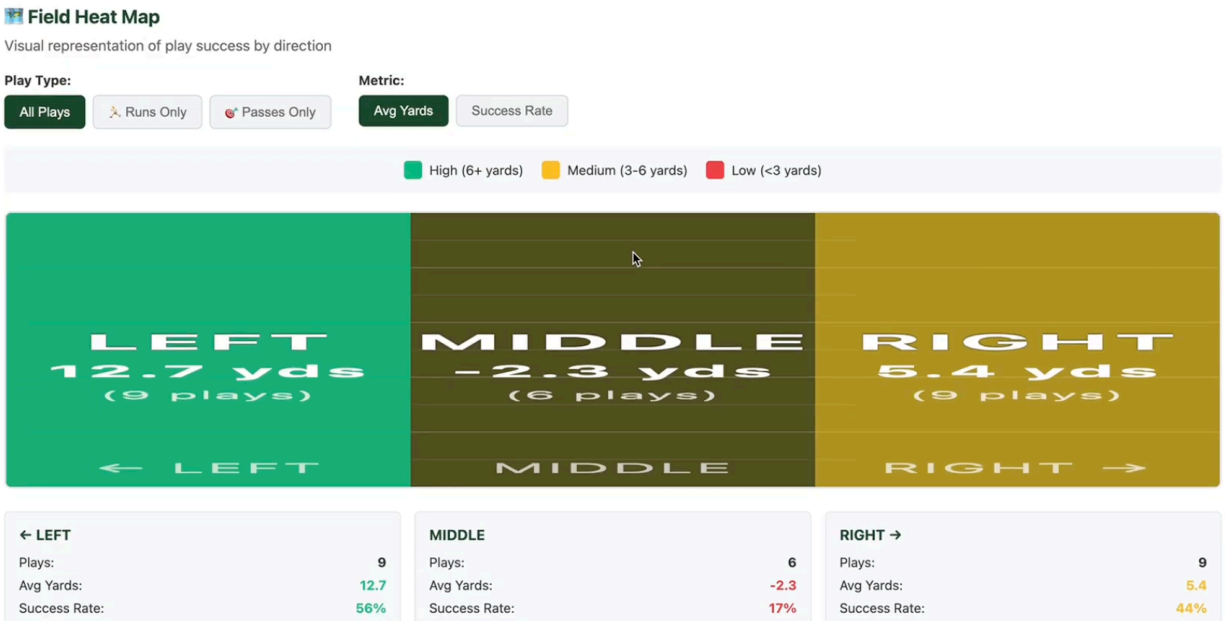

Figure 5: Field heat map showing success zones by direction with run/pass filtering

The Opponent Scouting view starts with a dropdown to select which opponent to analyze. Once selected, it displays cards showing total plays against that team, run/pass split, and most-used formation. Below that, a pie chart breaks down run versus pass calls, and a table shows down-by-down tendencies specifically for that opponent. As shown in Figure 6.

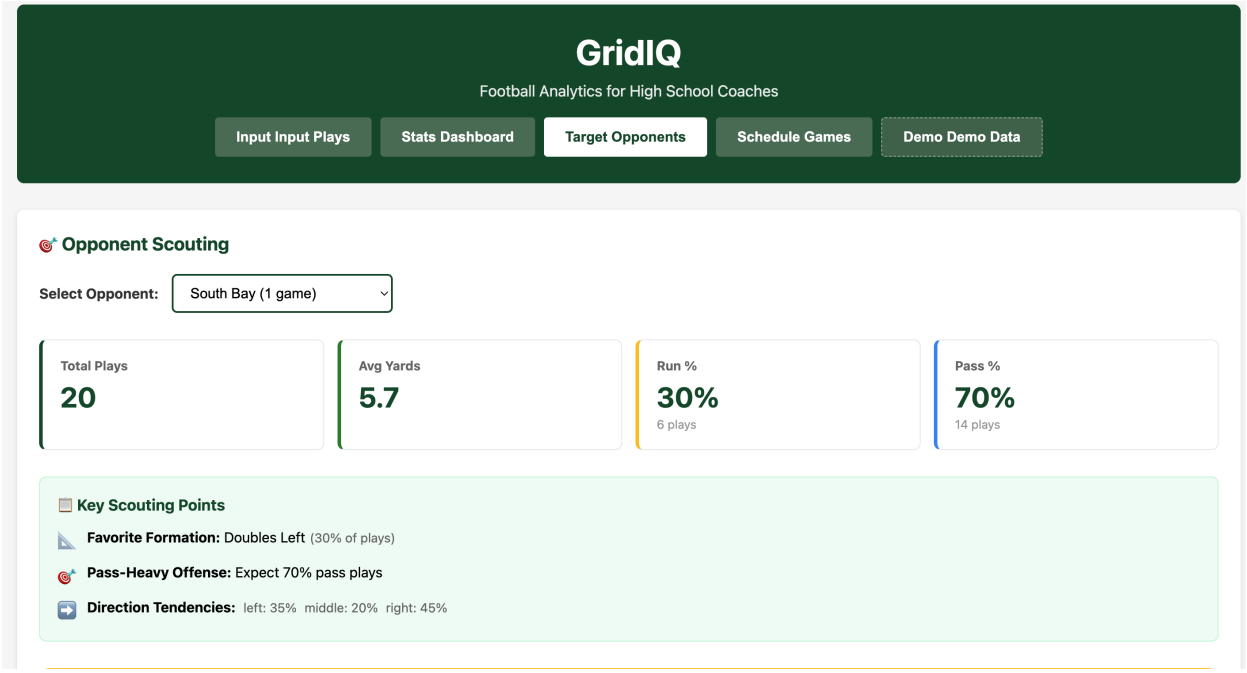

Figure 6: Opponent-specific analysis showing tendencies and play distribution

The Games Manager lists all scheduled and completed games as cards. Each card shows the opponent name, date, location, and final score if the game has been played. Win/loss is indicated by a green or red

border. Clicking “Add Game” opens a form to schedule a new matchup. Games without scores have an “Add Final Score” button that opens an inline editor. As shown in Figure 7.

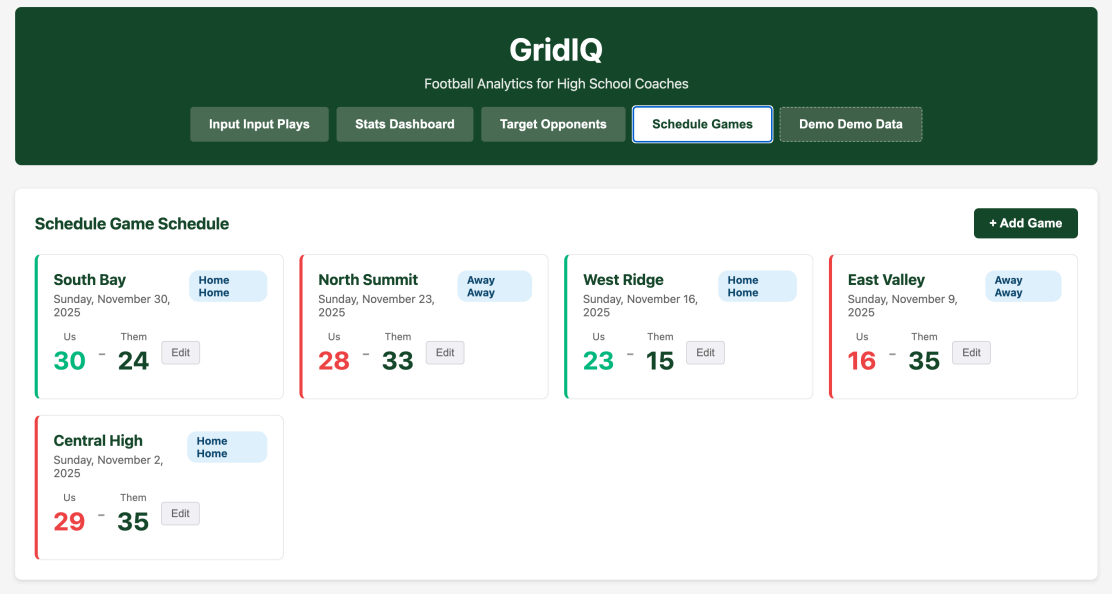

Figure 7: Game management interface with color-coded win/loss indicators

4.2. Backend API

The frontend communicates with the Flask backend through a REST API running on `localhost:5000`. All endpoints return JSON with a `success` boolean and either `data` or `error` fields. Table 2 shows the main API routes.

Method	Endpoint	Purpose	Returns
GET	/api/health	Check API status	Status message
GET	/api/plays	Get all plays	Array of play objects
POST	/api/plays	Create new play	Created play object
DELETE	/api/plays/:id	Delete play	Success message
GET	/api/stats/overview	Overall statistics	Total plays, yards, TDs
GET	/api/stats/by-down	Stats by down	Plays per down, avg yards
GET	/api/stats/by-formation	Stats by formation	Formation usage, success
GET	/api/stats/success-rate	Success rate analysis	Success % by down
POST	/api/predict	Predict play type	Prediction with confidence
POST	/api/predict/opponent/:name	Opponent prediction	Opponent-specific prediction
GET	/api/games	Get all games	Array of game objects
POST	/api/games	Create game	Created game object
PUT	/api/games/:id	Update game	Updated game object
GET	/api/opponents	List opponents	Opponent names and counts
GET	/api/opponent/:name/stats	Opponent stats	Detailed opponent analysis

Table 2: REST API endpoints for frontend-backend communication

The API uses standard HTTP status codes. Successful requests return `200 OK` or `201 Created`. Errors return `400 Bad Request` with an error message explaining what field is missing or a bad response. If a play is submitted without a formation, the response is `{"success": false, "error": "Missing required field: formation"}`. Data flows through the system in a request-response pattern. When a coach enters a play, the `PlayInputForm` component packages the data as JSON and sends a `POST` request to `/api/plays`. The backend validates the data, saves it to PostgreSQL using SQLAlchemy, and returns the created play object. The frontend then calls `fetchPlays()` to refresh the play list, and the `PlayList` component re-renders with the new data.

The following screens are planned for Phase 2 implementation and are current mockups on what we believe the features to look like.Using AI technologies to generate these screens and what we plan to have them look like.

4.3. Video Upload Screen

The video upload screen provides a simple drag-and-drop for coaches to add game footage. The screen centers around a large upload zone where users can either drag video files directly or click to open a file browser. The system accepts standard video formats. During upload, a progress bar shows completion percentage and estimated time remaining. Once the upload finishes, the interface automatically transitions to the video tagging screen where coaches can begin marking plays. As shown in Figure 8, the design keeps the interface clean and focused on the single task of getting video into the system.

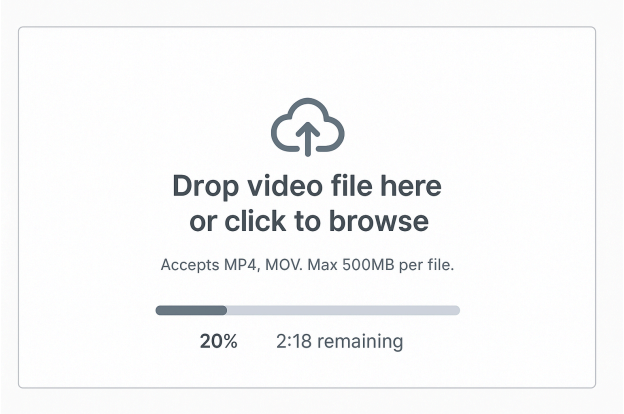

Figure 8: Wireframe for video upload interface with drag-and-drop zone

4.4. Video Tagging Interface

The video tagging screen uses a split-pane layout that lets coaches watch footage while logging plays. The video player occupies the left side of the screen and includes standard controls for playback, skipping forward or backward by 5 seconds, and adjusting playback speed. A timeline scrubber below the video shows thumbnail previews when hovering, making it easy to navigate to specific moments. The right side displays the tagging panel, which prominently shows the current video timestamp at the top. Quick-tag buttons for common formations like “Doubles Right” and “Trips Left” allow coaches to log a play with a single click at the current timestamp. As shown in Figure 9, this layout keeps both the video and tagging tools visible, eliminating the need to switch between screens.

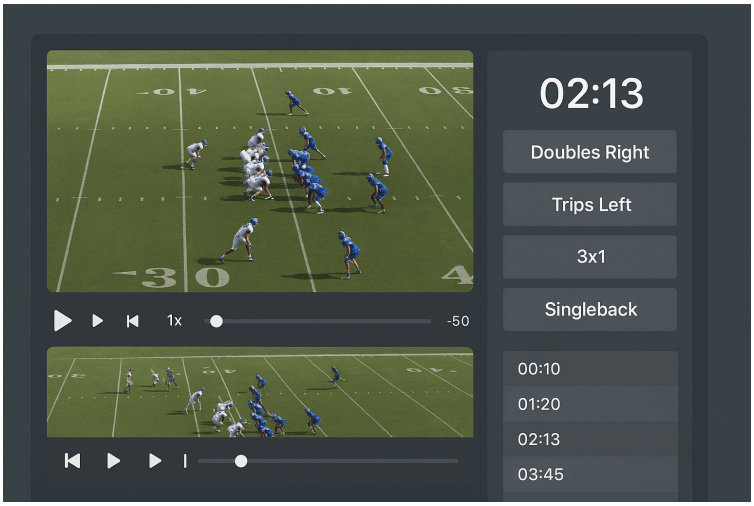

Figure 9: Wireframe for video tagging interface with split-pane layout showing player and tag controls

4.5. Video Review Screen

The video review interface appears as a modal overlay when a coach clicks on any play that has associated video footage. The modal divides into three horizontal sections. The top section shows the video player starting at the exact timestamp where the play was tagged.The middle section displays the formation diagram for the selected play with overlays showing the actual play data including yards gained, result, and direction arrows. The bottom section presents a detailed play information card containing all logged data such as formation, down, distance, and defensive coverage. An “Edit Tags” button opens the play

input form pre-filled with current data, allowing coaches to correct or enhance the information without leaving the video context. Figure 10 shows how this modal keeps video, visualization, and data all accessible in one view.

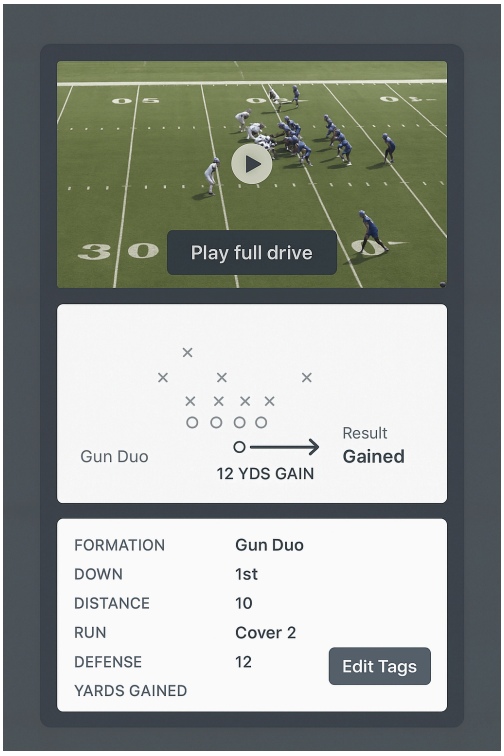

Figure 10: Wireframe for video review modal showing linked video playback with play data

5. Data model

GridIQ uses PostgreSQL [7] to store all project data. We went with Postgres over something lighter like SQLite[11] because we need specific features that SQLite does not handle well. The main reason is window functions and complex aggregations. When we calculate things like “3rd down success rate in the red zone compared to league average,” we are using SQL queries with with multiple CTEs. SQLite can technically do this, but it is way slower and does not optimize these queries nearly as well. Postgres also gives us better JSON support for storing flexible data like defensive alignments, and the EXPLAIN ANALYZE tools make it easier to debug slow queries. Since we are already running Docker containers, spinning up Postgres is not really more complicated than SQLite anyway.

The database has four main tables that connect through foreign keys. The Team table stores basic info about your team and opponents you face. The Game table tracks individual matchups with dates, scores, and locations. The Play table is where everything happens. It logs every snap with 25+ fields covering formation, result, defensive look, and context. The User table handles authentication, though that is not fully implemented yet.The relationships between these are illustrated in Figure 11.

Figure 11: Entity relationship diagram showing database tables and their relationships.

5.1. Video Storage

Videos do not go in the database because that would be a lot for file sizes. Instead, we store video files on the local file system in a `videos/` directory that Docker mounts as a volume. When a user uploads a video, the backend saves the file with a unique filename and stores just the path in the database. We will add a Videos table that looks like this. It includes an `id` as primary key, `team_id` as foreign key to teams, `game_id` as foreign key to games, `filename` for unique filename on disk, `original_name` for what the user called it, `file_size` in bytes, `duration` in seconds, `upload_date` as timestamp, and `processed` as boolean.Each play can link to a specific moment in a video through a `video_id` and `timestamp` field. So when you click on a play in the play list, it can jump to that exact moment in the video player. This keeps the video files separate from the structured data but still connects everything.

5.2. Backend Efficiency

Tendency analysis needs to be filtered by different contexts such as opponent, down, formation, and date range.Postgres is fast enough that aggregating 500 plays takes about 50ms. The queries use indexes on `team_id`, `formation`, `down`, and `created_at`, so they do not have to scan the whole table. If we ever get to thousands of plays and performance becomes an issue, we can add views or a cache layer. But for the scope of this project, which targets high school teams with about 1000 plays per season, the real time calculation is simpler.

5.3. Schema Details

The full schema is defined in `backend/models.py` using SQLAlchemy ORM. Here is what each table stores.

The Teams table contains basic team info (name, season). Right now we have one team per database instance. Multi-team support is future work.The Games table stores opponent name, game date and location, home or away designation, final scores, and weather and notes. The Plays table is the most detailed. It includes formation , play call, down and distance, yards gained , result , play type and direction, defensive alignment , success metrics , game context, and timestamps for sorting.The Users table holds email and hashed password, name, foreign key to team.The database uses foreign key constraints to enforce relationships. For example, you cannot create a play without a valid `team_id`, and if you delete a game, all its plays get set to `game_id = NULL` instead of being deleted. We use ON DELETE SET NULL for this. This prevents phantom records and keeps data integrity solid.

6. Testing

Testing for GridIQ was conducted across three categories: unit testing, integration testing, and user testing. Each category targets a different one of the systems, and together they make sure that the core requirements described in Table 1 are met. The results are summarized in Table 3 at the end of this section.

6.1. Unit Testing

Unit tests were written using pytest [12] for the Flask backend and React Testing Library [13] for the frontend. Backend unit tests cover the data validation logic, tendency detection calculations, and the play predictions. For example, one test passes a set of 20 sample plays where a single formation appears 85 percent of the time and checks that the tendency detector correctly identifies it as a pattern. Another test verifies that submitting a play with a down value outside the 1 to 4 range returns a 400 status code with the correct error message.

6.2. Integration Testing

Integration tests verify that the frontend, backend, and database work together correctly. Backend integration tests use pytest with a separate test database that is cleared before each test run. These tests simulate full request cycles, for example, submitting a POST request to /api/plays and then querying /api/stats/overview to confirm the new play is reflected in the totals. Integration tests also confirm that deleting a game sets the game_id on associated plays to NULL rather than removing the play records, which is the behavior described in the data model section.

6.3. User Testing

User testing was conducted with a group of teammates who walked through a scripted set of tasks. Each person was asked to create a new play entry, review the tendency dashboard, use the opponent scouting view to analyze a sample opponent, and export play data as a CSV file. The athletes were observed for hesitation and confusion, and brief notes were taken during each session.

Overall feedback was positive. The football players found the play input form straightforward and appreciated that the dropdown menus prevented typos. The tendency warning cards on the dashboard were described as easy to read and useful. The main piece of feedback was that the heat map needed a short explanation of what the color coding meant, as a few participants were unsure whether green represented high usage or high success. A label was added to clarify this. No participant required more than a few minutes to complete all tasks, which means the interface is approachable for users who are not technical.

Requirement	Unit	Integration	UI	API	User
Play input	100%	100%	✓	✓	✓
Play visualization	100%	100%	✓	✓	✓
Tendency dashboard	85%	85%	N/A	✓	✓
Data export	90%	85%	N/A	✓	✓
Performance and scalability	100%	80%	✓	✓	✓
Video upload and linking	N/A	N/A	N/A	N/A	N/A
Computer vision	N/A	N/A	N/A	N/A	N/A

Table 3: Testing results by requirement. N/A indicates the feature is not yet implemented.

As shown in Table 3, the five fully implemented requirements have all been tested across automated and user methods. Video upload and computer vision are not yet implemented and will be tested during the final phase of development in April 2026.

7. Implementation

The current prototype demonstrates the feasibility of the GridIQ system by implementing core functionality for play input, storage, and basic visualization. This section describes what has been completed, how it satisfies requirements, and what remains to be implemented during the Senior Capstone phase.

7.1. Implemented and Partially Implemented Features

Play Input System. The prototype includes a fully functional play input form implemented as a React component. Users can select formations from a drop down menu containing common offensive alignments. Also always you to input distance, yards gained or lost, and result type selected from a drop down. There is validation to make sure all required fields are completed before submission and displays error messages for invalid inputs such as down values outside the 1-4 range. This implementation fully satisfies the Play Input System requirement described in Section 2.

Database Storage. The prototype implements the PostgreSQL database schema described in Section 5 including the Teams, Users, Plays, and Playbook tables. Database migrations are managed using Alembic [14], a database migration tool for SQLAlchemy that tracks schema changes and enables version control of the database structure. The Flask backend includes SQLAlchemy models to each table and uses foreign key relationships. When users submit the play input form, the frontend makes a `POST` request to the `/api/plays` endpoint, which then checks the data and inserts a new row in the Plays table. The prototype demonstrates reliable data persistence by successfully storing and retrieving play records across multiple sessions.

Play List and Retrieval. The prototype includes a play list view that shows all plays entered by the user's team in reverse chronological order. Each row in the list shows formation, play call, down and distance, and result. Users can click on any play to view detailed information including yards gained and game situation. The backend `/api/plays` endpoint supports filtering by formation. This functionality demonstrates the data retrieval capabilities needed for tendency analysis and satisfies part of the Play Visualization requirement.

Formation Diagram Generation. The prototype implements a simplified version of play visualization. The visualization accepts a formation name as input and positions player on a football field diagram according to formation templates. For example, selecting "Doubles Right" positions two wide receivers on the right side of the formation, one tight end, and standard offensive line alignment. As shown in Figure 12, the diagram is static and does not include arrows showing play direction or player motion. This partial implementation demonstrates the feasibility of automated diagram generation but requires improvement to fully satisfy the Play Visualization requirement.

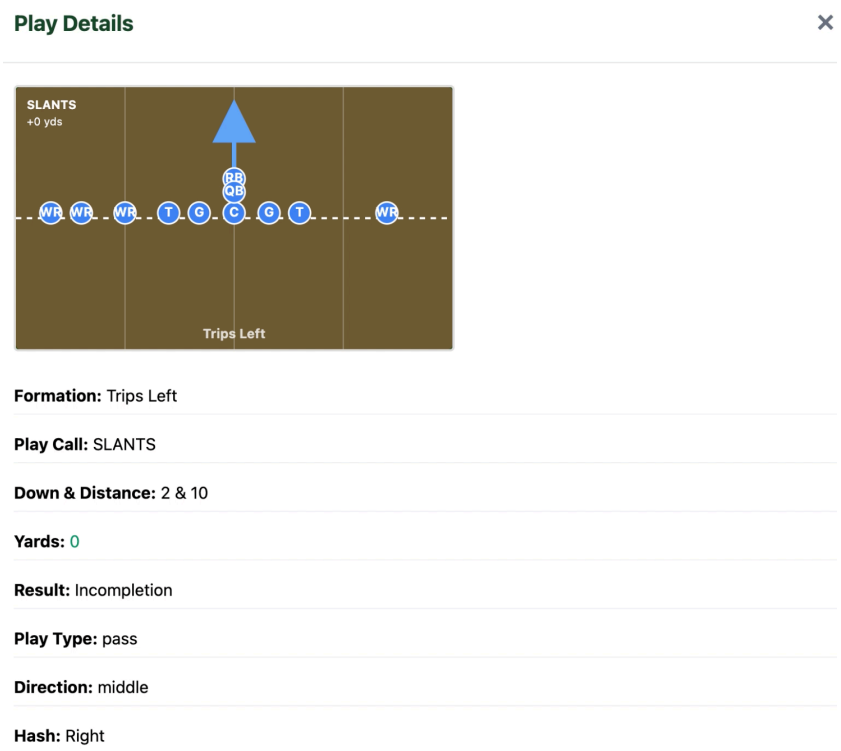

Figure 12: Formation Diagram Generation

CSV Export. The prototype includes a basic export function that generates CSV files containing play data. The backend `/api/export` endpoint returns a CSV file with headers including Game Date, Formation, Play Call, Down, Distance, Yards, and Result. The frontend makes a file download when users click the export button, as shown in Figure 13. This implementation fully satisfies the Data Export requirement for CSV format, though PDF report generation is not yet implemented.

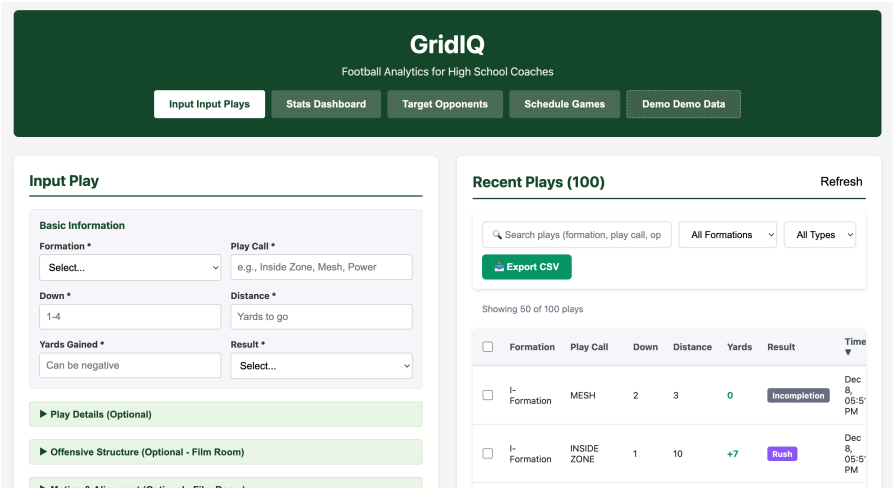

Figure 13: Export Button

Tendency Dashboard. As demonstrated in Figure 14, the prototype can retrieve and display raw play data, though it does not yet include the full statistical aggregation and charting capabilities described

in the Tendency Analysis Dashboard requirement.The Recharts library will be integrated to render bar charts and pie charts visualizing these statistics.

Figure 14: Dashboard Tab

Requirement	Completion
Play Input System	100%
Play Visualization	100%
Tendency Analysis Dashboard	100%
Data Export	100%
Usability and Accessibility	90%
Local Operation	100%
Performance Efficiency	75%
Data Security and Privacy	50%
Video Upload and Linking	50%
Scalability and Extensibility	25%
User Authentication and Data Management	25%

Table 4: Implementation completion status by requirement.

As shown in Table 4, the highest priority requirements are fully implemented. Computer vision integration remain as the primary focus for the remainder of the capstone timeline.Authentication and security are partially complete.

7.2. Features Not Yet Implemented

Video Upload and Tagging. The prototype does not include video upload functionality or the Video Tags database table. Implementing this feature requires involving object storage, which introduces more complexity that will be addressed during the Senior Capstone phase. The frontend does not include placeholder components for the video review screen.

Advanced Play Visualization. The prototype's formation diagrams are static and do not include directional arrows showing player motion, blocking assignments, or pass routes. Fully implementing the Play Visualization requirement requires creating an arrow drawing system that accepts play specific parameters and renders appropriate overlays on the formation diagram.

Computer Vision Integration. The architecture includes a future computer vision features but this component is not implemented in the prototype. Adding automated video analysis capabilities requires integrating YOLOv8 for player detection and implementing tracking algorithms to identify movement patterns. This represents a significant undertaking that will be the primary focus of the spring semester.

8. Timeline

The development timeline spans from January 2026 through April 2026, with major milestones aligned to the Senior Capstone schedule. The timeline prioritizes highest priority requirements first and allocates additional time for computer vision integration due to its complexity and risk.

January 2026: Video Infrastructure. January focuses on implementing video upload, storage, and basic playback functionality. The first two weeks involve setting up object storage using Amazon S3 for cloud hosting, configuring the Flask backend to generate. By the end of January, users will be able to upload game film and manually tag plays within videos, satisfying the Video Upload and Linking requirement.

February 2026: Computer Vision Foundation. February marks the beginning of computer vision integration, the most technically challenging component of the system. The first milestone involves researching existing football tracking datasets and pre-trained models. By the end of February, the infrastructure for running computer vision tasks asynchronously will be operational.

March 2026: Computer Vision Implementation. March focuses on implementing and refining the computer vision algorithms. The first two weeks involve integrating YOLOv8 for player detection in video frames. This includes writing preprocessing code to extract frames from uploaded videos, running inference on each frame to detect player bounding boxes, and storing detection results in the database.

April 2026: Testing, Refinement, and Documentation. April is reserved for testing, bug fixes, and final documentation. The first two weeks involve executing the full testing plan described in Section 6, including automated unit and integration tests, end-to-end tests, performance testing, and user testing sessions with coaches. Issues discovered during testing will be triaged and fixed based on severity. The final two weeks focus on code cleanup, documentation updates, and preparation of the final capstone presentation. This includes recording demonstration videos, writing user guides, and ensuring the system is deployable for actual use by a high school team during their upcoming season.

Timeline	Milestone
January 2026	Video upload, storage, and manual tagging
February 2026	Computer vision infrastructure and worker pool
March 2026	Player detection and formation identification
April 2026	Comprehensive testing, refinement, and documentation

Table 5: Development timeline with major milestones.

As shown in Table 5, the timeline front-loads core functionality and reserves March and April for computer vision features and testing. This approach ensures that even if computer vision integration proves more difficult than anticipated, the system will still provide value through manual play input, visualization, and tendency analysis.

9. AI Usage

Claude, developed by Anthropic [15], was used at several points during the completion of this project and document. This section describes specifically how it was used and where it was not. Claude was used to assist with debugging during development. When backend API endpoints returned non expected responses or SQLAlchemy had wrong results, Claude was used to help identify the source of the issue. In each case, the suggested fix was reviewed and tested manually before being applied to the codebase. All final implementation decisions remained with me. Claude was also used to assist with portions of this document. Specifically, it helped with phrasing and structure in the testing and implementation sections.Claude was used to refine wording and ensure the writing met the formatting requirements of the assignment.The wireframe images included in the User Interfaces section were generated using Claude. These wireframes represent planned features that are not yet implemented, specifically the video upload screen, video tagging screen, and video review modal. They are included to illustrate the intended design direction for Phase 2 development. Claude was used to write the testing code,it was not used to generate the system architecture, design the data model, or make any core technical decisions. All diagrams outside of the wireframes, including the entity relationship diagram and architecture diagrams, were created manually.

10. Summary

This proposal describes GridIQ, an intelligent football analytics and visualization system designed to make professional-level play analysis accessible to high school and amateur teams. The system addresses the need for affordable analytics tools by combining manual play data entry with automated formation diagrams and tendency analysis. Future computer vision capabilities will enable semi-automated video analysis to identify player positions and movement patterns. The requirements analysis identified eleven functional and non-functional requirements prioritized according to their importance for providing value to coaches. The highest-priority requirements focus on play input, visualization, and usability, ensuring the system can immediately support game planning and scouting workflows. Medium-priority requirements including tendency analysis and video linking extend the system's analytical capabilities, while lower-priority requirements address data export and advanced security features.The three-tier architecture separates the React frontend, Flask backend, and PostgreSQL database into independent components that communicate through well-defined REST APIs. This design supports both local deployment for teams concerned about data privacy and cloud hosting for teams preferring managed infrastructure. The architecture also includes future enhancements including computer vision processing.

The prototype demonstrates that core technical challenges are solvable by implementing play input, database storage, authentication, and basic formation diagrams. The remaining work involves building tendency analysis queries, enhancing visualizations, implementing video functionality, and integrating computer vision models. The development timeline allocates appropriate time for each milestone and prioritizes highest-risk components later in the schedule when foundational features are stable. Testing

will combine automated unit, integration, and end-to-end tests with user-based testing involving high school coaches. This multi-faceted approach ensures both technical correctness and real-world usability. The testing plan explicitly maps each requirement to specific testing strategies and success criteria. GridIQ represents an opportunity to apply modern web development, database design, and machine learning techniques to solve a practical problem facing grassroots football programs. By the end of Senior Capstone in April 2026, the system will provide coaches with tools to visualize plays, identify tendencies, and analyze game film more efficiently than manual methods allow. The project demonstrates feasibility through a working prototype and a detailed implementation plan that accounts for technical complexity and timeline constraints.

References

[1] Hudl, "Hudl Assist: Automated Sports Video Analysis." Accessed: Nov. 20, 2025. URL: https://www.hudl.com/products/assist

[2] Catapult Sports, "Catapult Sports: Player Tracking Technology." Accessed: Nov. 20, 2025. URL: https://www.catapultsports.com/

[3] National Football League, "NFL Next Gen Stats." Accessed: Nov. 20, 2025. URL: https://nextgenstats.nfl.com/

[4] Ultralytics, *YOLOv8*. Version 8.0.0. Jan. 10, 2023. URL: https://github.com/ultralytics/ultralytics

[5] I. Meta Platforms, *React*. Version 18.2.0. June 14, 2022. URL: https://reactjs.org/

[6] Pallets Projects, *Flask*. Version 3.0.0. Sept. 30, 2023. URL: https://flask.palletsprojects.com/

[7] PostgreSQL Global Development Group, *PostgreSQL*. Version 16.0. Sept. 14, 2023. URL: https://www.postgresql.org/

[8] I. Docker, *Docker*. Version 24.0. Apr. 19, 2023. URL: https://www.docker.com/

[9] Microsoft, *TypeScript*. Version 5.2.0. Aug. 24, 2023. URL: https://www.typescriptlang.org/

[10] SQLAlchemy Contributors, *SQLAlchemy*. Version 2.0.0. Jan. 26, 2023. URL: https://www.sqlalchemy.org/

[11] SQLite Development Team, *SQLite*. Version 3.45.0. Jan. 15, 2024. URL: https://www.sqlite.org/

[12] Holger Krekel and pytest-dev team, *pytest*. Version 7.4.0. July 04, 2023. URL: https://pytest.org/

[13] Kent C. Dodds and contributors, *React Testing Library*. Version 14.0.0. May 03, 2023. URL: https://testing-library.com/react

[14] SQLAlchemy Contributors, *Alembic*. Version 1.12.0. Sept. 22, 2023. URL: https://alembic.sqlalchemy.org/

[15] Anthropic, *Claude AI*. Version 1.12.0. Sept. 22, 2023. URL: https://www.anthropic.com/